

EduGenie: Google Gemini Powered Learning Assistant

Introduction

EduGenie is an AI-based educational assistant developed to help students understand topics, get answers, practice through quizzes, summarize study materials, and get learning recommendations. Designed for students of all academic levels, EduGenie enables users to:

- Ask questions and receive smart, concise answers
- Understand complex concepts through simplified explanations
- Generate quizzes from topics or text
- Receive personalized learning recommendations
- Summarize large educational passages

Problem Statement

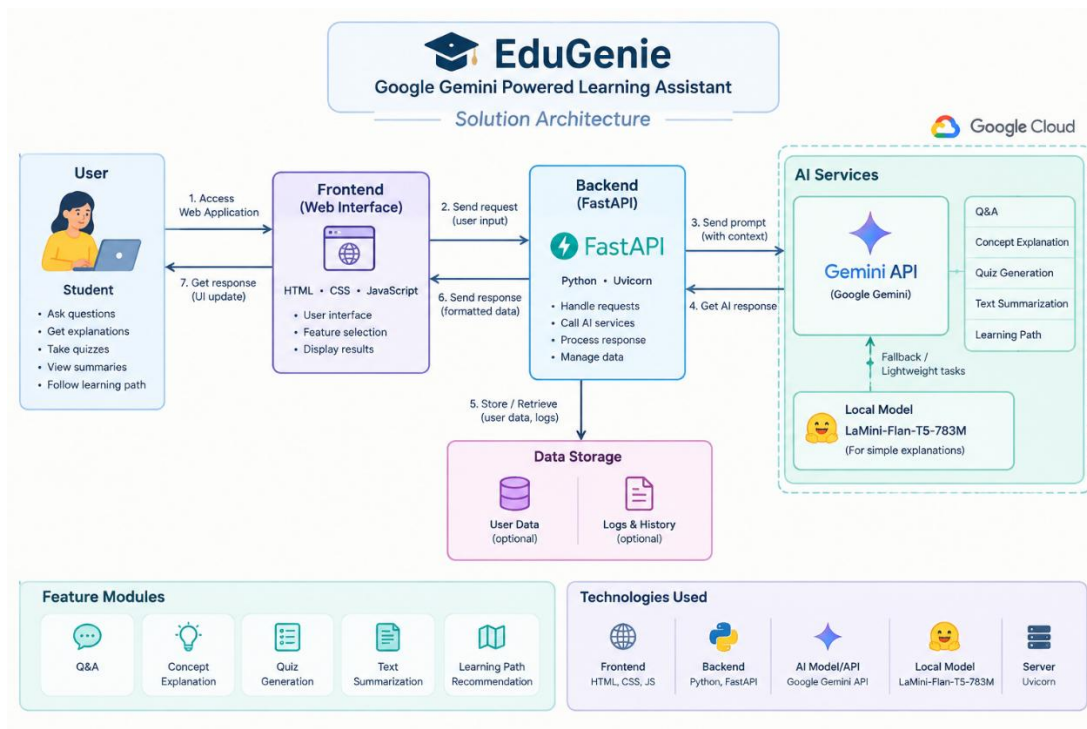
Students often find it difficult to understand complex topics, get quick answers, practice questions, revise study materials, and decide what to learn next. To solve these problems, we developed EduGenie.

Objective

The main objective of our project is to provide multiple learning features in a single AI-powered platform. Like,

- Question and Answer
- Concept Explanation
- Quiz Generation
- Text Summarization
- Learning Path Recommendation

Architecture



Pre-requisites

1. Python 3.10+

Python is required for developing and running the EduGenie backend application. In this project, Python is used to implement the FastAPI backend and integrate the AI services.

2. FastAPI Framework

FastAPI is used to develop the backend APIs of the EduGenie application. It handles user requests, processes inputs, and communicates with the Google Gemini API.

3. HTML & CSS

HTML and CSS are used to create the frontend interface of the application. HTML provides the structure of the web pages, while CSS is used for styling and layout.

4. Google Gemini API Key

A Google Gemini API key is required to connect the application with the Gemini AI model. The API is used to generate answers, explanations, quizzes, summaries, and learning path recommendations.

5. Uvicorn

Uvicorn is used as the ASGI server to run the FastAPI application locally.

6. Jinja2

Jinja2 is used as the HTML templating engine to connect the FastAPI backend with the HTML frontend templates.

Project Workflow

MILESTONE 1: Model Selection and Architecture

AI Models

Gemini 3.5 Flash Lite (via API):

Used for: Question and Answer, Concept Explanation, Quiz Generation, Text Summarization, and Learning Path Recommendation.

Benefits: Provides fast AI-generated responses and supports multiple educational tasks through cloud-based inference.

LaMini-Flan-T5-783M (Local):

Used for: Concept Explanation.

Status: Included as an optional local explanation model, but currently disabled in the project configuration.

Benefits: Lightweight and suitable for local inference.

Folder Architecture:

EduGenie/		
1.	main.py	# FastAPI app
2.	explanation_module.py	# Concept explanation logic
3.	qna.py	# Question answering
4.	quiz_module.py	# Quiz generation
5.	summary_module.py	# Summarization
6.	learning_path.py	# Learning recommendations
7.	templates/index.html	# HTML frontend

8.	static/style.css	# Styling
9.	requirements.txt	# Python dependencies

Epic 2: Core Functionalities Development

Activity 2.1: Module Implementation

1. Explanation Module

The Explanation Module helps students understand difficult topics in simple language. It uses the **Google Gemini 3.5 Flash Lite** model to generate clear and easy-to-understand explanations based on the topic given by the student. This feature is useful for students who need quick explanations while learning.

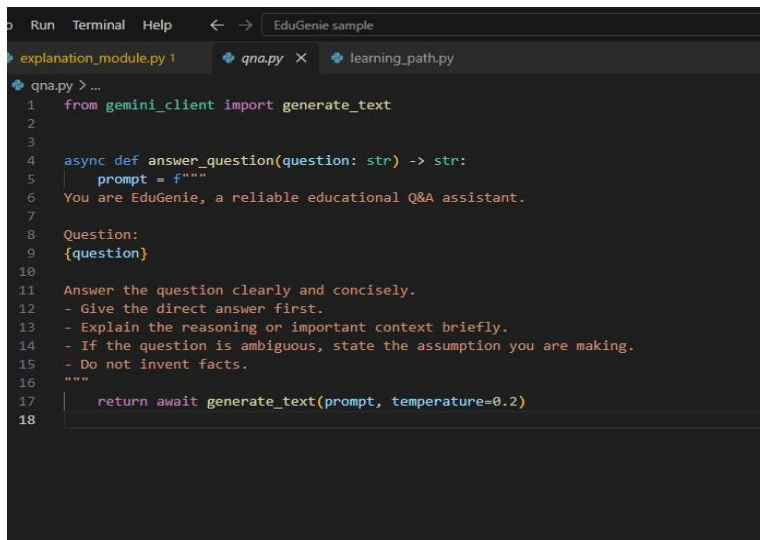
```

explanation_module.py 1 X
1 import os
2 from gemini_client import generate_text
3 async def _local_explanation(topic: str) -> str | None:
4     """Optional LaMini-Flan-T5 explanation. Disabled by default to keep setup lightweight."""
5     if os.getenv("USE_LOCAL_EXPLAINER", "false").lower() != "true":
6         return None
7     try:
8         from transformers import pipeline
9         model_name = os.getenv("LOCAL_EXPLAINER_MODEL", "MBZUAI/LaMini-Flan-T5-783M")
10        generator = pipeline("text2text-generation", model=model_name)
11        prompt = (
12            "Explain this educational topic for a beginner in simple language. "
13            "Use a short definition, 3-5 key points, and one simple example. "
14            f"Topic: {topic}"
15        )
16        result = generator(prompt, max_new_tokens=220, do_sample=False)
17        return result[0]["generated_text"].strip()
18    except Exception:
19        return None
20 async def explain_topic(topic: str) -> str:
21     local_result = await _local_explanation(topic)
22     if local_result:
23         return local_result
24     prompt = f"""
25     You are EduGenie, an educational assistant.
26     Explain the following topic for a beginner.
27     Topic: {topic}
28     Requirements:
29     - Use very simple language.
30     - Start with a one-sentence definition.
31     - Give 3 to 5 important points.
32     - Include one simple example or analogy.
33     - Avoid unnecessary jargon.
34     """
35     return await generate_text(prompt, temperature=0.2)
36

```

2. QnA Module

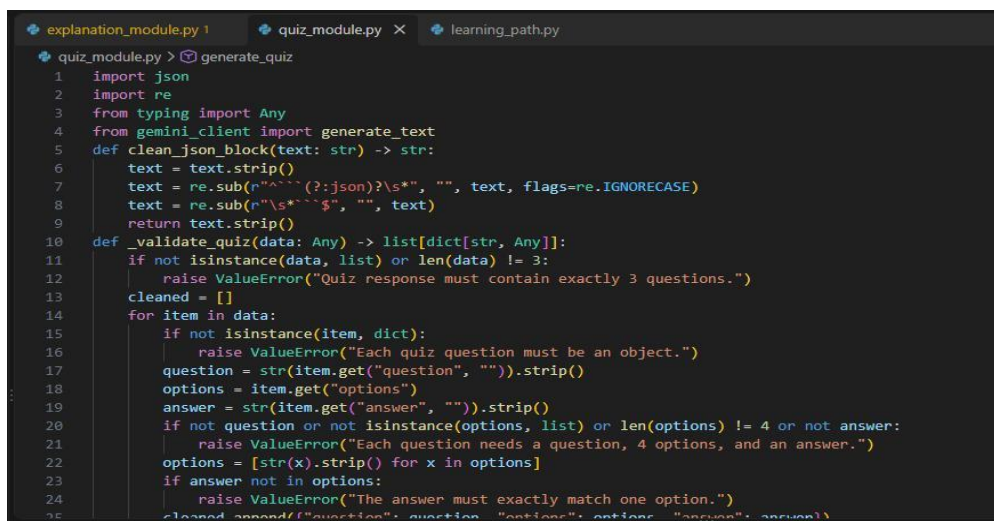
The QnA Module allows students to ask questions related to their studies. The question is sent from the frontend to the **FastAPI backend**, which communicates with the **Google Gemini API**. Gemini generates the answer and the response is displayed on the frontend. This helps students get quick answers to their doubts.



```
Run Terminal Help < -> EduGenie sample
explanation_module.py 1 qna.py X learning_path.py
qna.py > ...
1 from gemini_client import generate_text
2
3
4 async def answer_question(question: str) -> str:
5     prompt = f"""
6     You are EduGenie, a reliable educational Q&A assistant.
7
8     Question:
9     {question}
10
11     Answer the question clearly and concisely.
12     - Give the direct answer first.
13     - Explain the reasoning or important context briefly.
14     - If the question is ambiguous, state the assumption you are making.
15     - Do not invent facts.
16     """
17     return await generate_text(prompt, temperature=0.2)
18
```

3. Quiz Module

The Quiz Module generates **multiple-choice questions (MCQs)** based on the topic or content provided by the student. The Gemini model processes the given input and generates relevant questions with answer options. This feature helps students practice and check their understanding of a topic.



```
explanation_module.py 1 quiz_module.py X learning_path.py
quiz_module.py > generate_quiz
1 import json
2 import re
3 from typing import Any
4 from gemini_client import generate_text
5 def clean_json_block(text: str) -> str:
6     text = text.strip()
7     text = re.sub(r"```(?:json)?\s*", "", text, flags=re.IGNORECASE)
8     text = re.sub(r"\s```$", "", text)
9     return text.strip()
10 def _validate_quiz(data: Any) -> list[dict[str, Any]]:
11     if not isinstance(data, list) or len(data) != 3:
12         raise ValueError("Quiz response must contain exactly 3 questions.")
13     cleaned = []
14     for item in data:
15         if not isinstance(item, dict):
16             raise ValueError("Each quiz question must be an object.")
17         question = str(item.get("question", "")).strip()
18         options = item.get("options")
19         answer = str(item.get("answer", "")).strip()
20         if not question or not isinstance(options, list) or len(options) != 4 or not answer:
21             raise ValueError("Each question needs a question, 4 options, and an answer.")
22         options = [str(x).strip() for x in options]
23         if answer not in options:
24             raise ValueError("The answer must exactly match one option.")
25         cleaned.append({"question": question, "options": options, "answer": answer})
26
```

4. Summary Module

The Summary Module helps students revise long study materials quickly. It sends the given content to Gemini and generates a **short and meaningful summary** while keeping the important information. This makes revision easier and saves time.

```
explanation_module.py 1 quiz_module.py summary_module.py X learning_path.py
summary_module.py > ...
1 from gemini_client import generate_text
2
3
4 async def summarize_text(text: str) -> str:
5     prompt = f"""
6     Summarize the following educational passage for quick revision.
7
8     Requirements:
9     - Keep the important facts and relationships.
10    - Remove repetition and unnecessary detail.
11    - Use simple language.
12    - Prefer a short paragraph followed by bullet points when helpful.
13    - Do not add information that is not supported by the passage.
14
15    Passage:
16    {text}
17    """
18    return await generate_text(prompt, temperature=0.2)
19
```

5. Learning Path Module

The Learning Path Module provides a **step-by-step learning path** for a selected topic. Gemini generates learning recommendations starting from basic concepts and gradually moving towards advanced concepts. This helps students understand what they should learn first and what they can learn next.

```
explanation_module.py 1 learning_path.py X
learning_path.py > get_learning_recommendations
1 from gemini_client import generate_text
2 async def get_learning_recommendations(topic: str) -> str:
3     prompt = f"""
4     Create a personalized learning path for the topic: {topic}
5     Assume the learner is a beginner and wants to progress to an advanced level.
6     Structure the response as:
7     1. Goal
8     2. Beginner level
9     3. Intermediate level
10    4. Advanced level
11    5. Suggested timeline
12    6. Practice projects or exercises
13    7. Learning resources
14    For resources, suggest resource TYPES and well-known platforms/books/videos where
15    appropriate, but do not invent specific URLs. Keep the plan practical and step-by-step.
16    """
17    return await generate_text(prompt, temperature=0.35)
18
```

Activity 2.2: Backend API with FastAPI

FastAPI is used to create the backend APIs for the EduGenie application. Separate endpoints are created for each main feature. Each endpoint connects the user request to the corresponding module and returns the generated response.

API Endpoints

- /qa – Handles Question and Answer
- /explain – Handles Concept Explanation
- /quiz – Generates Quiz Questions
- /summarize – Summarizes study content

- /learn/recommendations – Generates Learning Path Recommendations

The backend receives the input from the frontend, sends it to the required module, and returns the result to the user interface.

```

explanation_module.py 1 quiz_module.py 2 main.py X 3 learning_path.py
main.py > ...
1 import os
2 from typing import Any
3
4 from dotenv import load_dotenv
5 from fastapi import FastAPI, Request
6 from fastapi.responses import HTMLResponse
7 from fastapi.staticfiles import StaticFiles
8 from fastapi.templating import Jinja2Templates
9
10 from explanation_module import explain_topic
11 from qna import answer_question
12 from quiz_module import generate_quiz
13 from summary_module import summarize_text
14 from learning_path import get_learning_recommendations
15
16 load_dotenv()
17
18 app = FastAPI(title="EduGenie - Google Gemini Powered Learning Assistant", version="1.0.0")
19 app.mount("/static", StaticFiles(directory="static"), name="static")
20 templates = Jinja2Templates(directory="templates")
21
22 @app.get("/", response_class=HTMLResponse)
23 async def home(request: Request):
24     return templates.TemplateResponse(
25         request=request,
26         name="index.html",
27         context={}
28     )
29
30 @app.get("/health")
31 async def health() -> dict[str, str]:
32     return {"status": "ok", "service": "EduGenie"}
33
34 @app.post("/qa")
35 async def qa(payload: dict[str, Any]):
36     question = str(payload.get("text", "")).strip()
37     ...

```

```

explanation_module.py 1 quiz_module.py 2 main.py X 3 learning_path.py
main.py > 4 learning_recommendations
36 async def qa(payload: dict[str, Any]):
37     question = str(payload.get("text", "")).strip()
38     if not question:
39         return {"ok": False, "error": "Please enter a question."}
40     try:
41         return {"ok": True, "result": await answer_question(question)}
42     except Exception as exc:
43         return {"ok": False, "error": str(exc)}
44
45 @app.post("/explain")
46 async def explain(payload: dict[str, Any]):
47     topic = str(payload.get("text", "")).strip()
48     if not topic:
49         return {"ok": False, "error": "Please enter a topic."}
50     try:
51         return {"ok": True, "result": await explain_topic(topic)}
52     except Exception as exc:
53         return {"ok": False, "error": str(exc)}
54
55 @app.post("/quiz")
56 async def quiz(payload: dict[str, Any]):
57     text = str(payload.get("text", "")).strip()
58     if not text:
59         return {"ok": False, "error": "Please enter a topic or passage."}
60     try:
61         return {"ok": True, "result": await generate_quiz(text)}
62     except Exception as exc:
63         return {"ok": False, "error": str(exc)}
64
65 @app.post("/summarize")
66 async def summarize(payload: dict[str, Any]):
67     text = str(payload.get("text", "")).strip()
68     if not text:
69         return {"ok": False, "error": "Please enter text to summarize."}
70     try:
71         return {"ok": True, "result": await summarize_text(text)}
72     except Exception as exc:
73         return {"ok": False, "error": str(exc)}
74
75 @app.post("/learn/recommendations")
76 async def learning_recommendations(payload: dict[str, Any]):
77     topic = str(payload.get("text", "")).strip()
78     if not topic:
79         return {"ok": False, "error": "Please enter a topic."}
80     try:
81         return {"ok": True, "result": await get_learning_recommendations(topic)}
82     except Exception as exc:

```

Epic 3: Frontend Development

Activity 3.1: Build Web Interface

HTML Form (index.html)

The EduGenie frontend is designed using HTML to provide a simple and user-friendly interface. The form contains the following options:

- **Task Dropdown:** Explain, QnA, Quiz, Summary, Recommend Path

- **Text Area:** Allows students to enter their question or study content.
- **Submit Button:** Sends the entered input to the backend for processing.
- **Result Container:** Displays the generated AI response.

CSS Styling (style.css)

CSS is used to improve the appearance and usability of the web interface. The design includes:

- Responsive layout for different screen sizes
- Styled buttons and input areas
- Clear task selection
- Proper spacing and alignment
- Separate result container for displaying responses

Activity 3.2: Live Integration

The frontend is connected to the FastAPI backend for live processing. When the student selects a task, enters the required input, and clicks the **Submit** button, the frontend sends a **POST request** to the corresponding FastAPI endpoint.

The backend processes the request using the selected EduGenie module and Google Gemini 3.5 Flash Lite. The generated response is then returned to the frontend and displayed below the input area.

Epic 4: Deployment

Activity 4.1: Run Locally

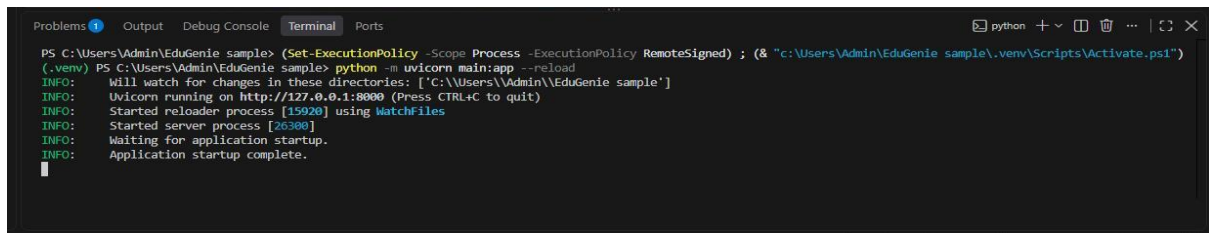
The EduGenie application is deployed and tested on a local system using **Uvicorn**, an ASGI server for running the FastAPI application.

The application can be started using:

```
python -m uvicorn main:app --reload
```

After starting the server, the application can be accessed through the browser at:

<http://127.0.0.1:8000>



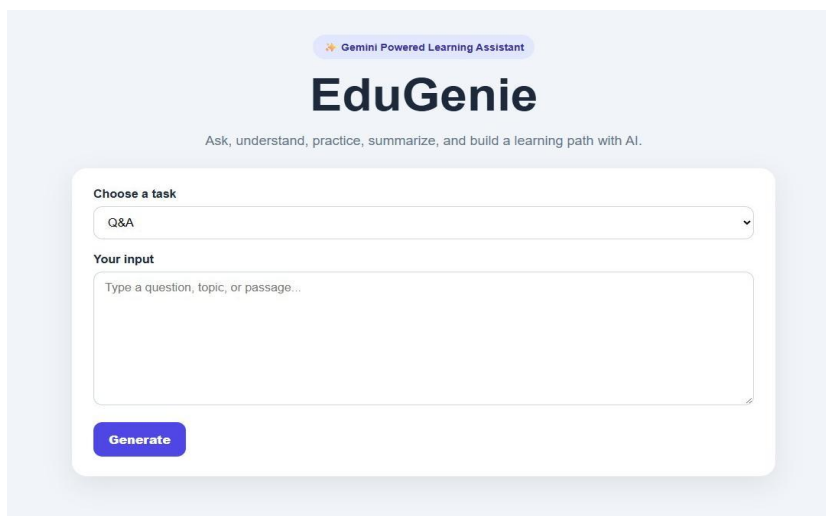
```
PS C:\Users\Admin\EduGenie sample> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:\Users\Admin\EduGenie sample\.venv\Scripts\Activate.ps1")
(.venv) PS C:\Users\Admin\EduGenie sample> python -m uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\Users\Admin\EduGenie sample']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [15920] using WatchFiles
INFO: Started server process [26300]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Activity 4.2: Functional Testing

Functional testing is performed to verify that all major EduGenie features work correctly. The following functions are tested:

1. **Ask a Question** – Enter a question and receive an AI-generated answer.
2. **Get an Explanation** – Enter a topic and receive a simple explanation.
3. **Generate a Quiz** – Enter a topic and generate quiz questions.
4. **Summarize Content** – Enter study content and receive a concise summary.
5. **Get a Personalized Learning Plan** – Enter a topic and receive a step-by-step learning path.

These tests verify the connection between the **frontend**, **FastAPI backend**, **feature modules**, and **Google Gemini API**.



a. Ask a question

Choose a task

Q&A

Your input

What is Artificial Intelligence?

Generate

Completed successfully.

Direct Answer:

Artificial Intelligence (AI) refers to the simulation of human intelligence processes by computer systems and machines.

Reasoning and Context:

These processes include learning (acquiring information and rules for using it), reasoning (using rules to reach approximate or definite conclusions), problem-solving, perception, and language understanding. AI is typically categorized into "narrow AI," which is designed to perform a specific task (like facial recognition or internet searches), and "artificial general intelligence" (AGI), which would possess the ability to understand, learn, and apply knowledge across any intellectual task like a human (though AGI remains theoretical).

b. Explain a topic

Gemini Powered Learning Assistant

EduGenie

Ask, understand, practice, summarize, and build a learning path with AI.

Choose a task

Explain a Topic

Your input

Explain Machine Learning in simple words

Generate

Completed successfully.

Generate

Completed successfully.

Hello! I am EduGenie, and I am here to help you learn.

Machine learning is a way to teach computers how to learn from experience, just like humans do, without needing a human to write exact rules for everything.

Here are 4 important points to understand it better:

1. ****Learning from Data:**** Instead of programming a computer with strict rules, we feed it a lot of information (called "data").

2. ****Finding Patterns:**** The computer looks at the data and tries to find patterns or habits in it.

3. ****Making Predictions:**** Once it finds the pattern, the computer can make smart guesses or decisions about new things it has never seen before.

4. ****Getting Better Over Time:**** The more information you give it, the smarter and more accurate it becomes.

****A Simple Analogy:****

Think about teaching a toddler what a dog is. You don't give the child a textbook definition (like "a four-legged furry mammal with a tail"). Instead, you just show them lots of pictures of dogs and say "dog." Soon, the child learns to recognize a dog on their own, even if it is a breed they have never seen before. Machine learning works the exact same way, but with computers!

c. Generate a Quiz

 Gemini Powered Learning Assistant

EduGenie

Ask, understand, practice, summarize, and build a learning path with AI.

Choose a task

Generate Quiz

Your input

Generate a quiz on Python programming

Generate

Completed successfully.

Completed successfully.

Quiz

1. Which of the following is the correct file extension for Python files?

☐ A. .pt

☐ B. .pyt

☒ C. .py

☐ D. .python

Check Answer

✓ Correct!

2. Which keyword is used to define a function in Python?

☒ A. function

☐ B. def

☐ C. define

☐ D. fun

Check Answer

✗ Not quite. Correct answer: def

3. Which of the following data types is immutable in Python?

☐ A. List

d. Summarize content

Choose a task

Summarize

Your input

Cloud computing is the delivery of computing services such as servers, storage, databases, networking, and software over the internet. It allows users to access resources without maintaining physical hardware. Cloud services are generally provided through models such as Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS). Cloud computing offers benefits such as scalability, flexibility, cost savings, and easy access to resources. It is widely used by businesses, educational institutions, and individuals for storing data, hosting applications, and providing online services.

Generate

Completed successfully.

Quick Revision Summary

Cloud computing delivers internet-based computing services, allowing users to access resources without managing physical hardware.

Core Services: Servers, storage, databases, networking, and software.

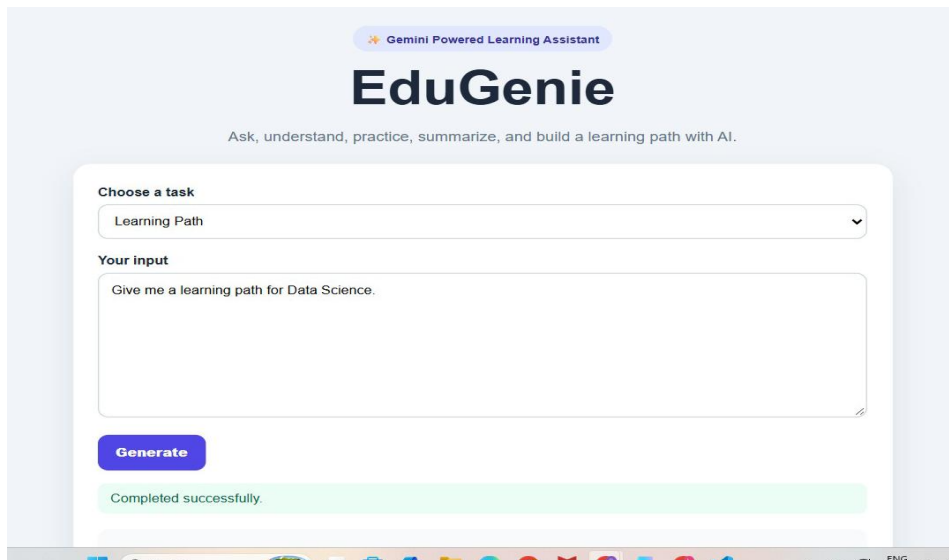
Service Models: Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS).

Key Benefits: Scalability, flexibility, cost savings, and easy resource access.

Common Uses: Storing data, hosting applications, and providing online services for businesses, schools, and individuals.

Search - Chattooga River, USA

e. Get a personalised learning path



Here is a comprehensive, step-by-step personalized learning path for Data Science, designed to take you from absolute beginner to advanced practitioner.

1. Goal

To build a strong foundation in programming, statistics, and mathematics; master data manipulation, visualization, and machine learning algorithms; and develop the ability to deploy complex predictive models to solve real-world business and research problems.

2. Beginner Level

Focus: Foundations of programming, basic math, and data literacy.

***Python Programming:** Learn syntax, variables, loops, functions, and object-oriented programming (OOP).

***Version Control:** Learn Git and GitHub for tracking code changes and collaboration.

***SQL & Databases:** Master writing queries ('SELECT', 'WHERE', 'JOIN', 'GROUP BY') to extract data from relational databases.

***Basic Mathematics & Statistics:** Understand descriptive statistics (mean, median, mode, variance, standard deviation) and basic probability.

***Data Manipulation & Visualization (Basics):** Learn to use Python libraries like Pandas and NumPy for data cleaning, and Matplotlib and Seaborn for basic charting.

3. Intermediate Level

Focus: Advanced data wrangling, exploratory data analysis (EDA), and core Machine Learning.

***Advanced SQL & Database Management:** Learn window functions, common table expressions (CTEs), and database optimization.

***Exploratory Data Analysis (EDA):** Master techniques to find patterns, spot anomalies, test hypotheses, and handle missing data.

***Supervised Machine Learning:** Learn algorithms like Linear Regression, Logistic Regression, Decision Trees, Random Forests, and Support Vector Machines (SVMs) using Scikit-Learn.

***Unsupervised Machine Learning:** Learn clustering (K-Means, Hierarchical) and dimensionality reduction

4. Advanced Level

Focus: Deep Learning, Big Data, MLOps, and production deployment.

* **Deep Learning:** Understand Neural Networks, Convolutional Neural Networks (CNNs) for computer vision, and Recurrent Neural Networks (RNNs) or Transformers for Natural Language Processing (NLP) using TensorFlow or PyTorch.

* **Big Data Engineering:** Learn how to process massive datasets using Apache Spark (via PySpark).

* **MLOps & Model Deployment:** Learn how to take models out of Jupyter Notebooks and put them into production. Cover API creation (FastAPI/Flask), Docker containerization, cloud deployment (AWS/GCP/Azure), and model monitoring.

* **Advanced Statistics & Experimentation:** Master A/B testing, causal inference, and advanced experimental design.

5. Suggested Timeline

Total Duration: ~10 to 12 months (assuming 10–15 hours of study per week).

* **Months 1–3:** Beginner Level (Python, SQL, Math/Stats, Git)

* **Months 4–7:** Intermediate Level (EDA, Pandas, Scikit-Learn, Supervised/Unsupervised ML)

* **Months 8–10:** Advanced Level (Deep Learning, Big Data, MLOps)

* **Months 11–12:** Capstone Projects, Portfolio Building, and Job/Promotion Preparation

6. Practice Projects or Exercises

* **Beginner:**

* *Titanic Survival Prediction:* Clean the dataset and use basic classification.

* *EDA on Netflix/Spotify Datasets:* Visualize trends in genres, release years, and popularity.

* **Intermediate:**

* *House Price Prediction:* Build a regression model using feature engineering and hyperparameter tuning.

* *Customer Churn Predictor:* Build a classification pipeline to predict whether bank customers will leave.

* **Advanced:**

* *End-to-End Deep Learning App:* Build an image classification app, containerize it with Docker, and deploy it to a cloud service.

*Titanic Survival Prediction: Clean the dataset and use basic classification.

* *EDA on Netflix/Spotify Datasets:* Visualize trends in genres, release years, and popularity.

* **Intermediate:**

* *House Price Prediction:* Build a regression model using feature engineering and hyperparameter tuning.

* *Customer Churn Predictor:* Build a classification pipeline to predict whether bank customers will leave.

* **Advanced:**

* *End-to-End Deep Learning App:* Build an image classification app, containerize it with Docker, and deploy it to a cloud service.

* *Recommendation System:* Build a collaborative filtering movie or product recommendation engine using Spark or deep learning.

7. Learning Resources

* **Interactive Coding Platforms:**

* *DataCamp* or *Dataquest* (Great for bite-sized, interactive coding practice)

* *LeetCode* / *HackerRank* (For SQL and Python practice)

* **Structured Online Courses:**

* *Coursera* (Look for "Python for Everybody" by Charles Severance, and the "Machine Learning Specialization" by Andrew Ng)

* *Udemy* (Look for popular comprehensive Bootcamp courses by instructors like Jose Portilla or Andrei Neagoie)

* **Books:**

* *"Python for Data Analysis"* by Wes McKinney (Creator of Pandas)

* *"Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow"* by Aurélien Géron (The industry bible for practical ML)

* *"An Introduction to Statistical Learning"* by Gareth James et al. (For the mathematical theory behind ML)

* **Video Channels (Free):**

* *StatQuest with Josh Starmer* (Best for intuitively understanding complex math and ML algorithms)

* *freeCodeCamp.org* (Full-length courses on Python, Pandas, SQL, and Machine Learning)

* *Krish Naik* (Practical industry insights and end-to-end project walkthroughs)

Conclusion

EduGenie has been developed as a lightweight AI-powered educational assistant that helps students learn in a simple and interactive way. The system provides

Question and Answer, Concept Explanation, Quiz Generation, Text Summarization, and Personalized Learning Path features through a web-based interface.

The project uses HTML and CSS for the frontend, Python and FastAPI for the backend, and Google Gemini 3.5 Flash Lite for AI-based responses. The modular design makes it easy to maintain and add new features in the future.

During development, the project helped demonstrate the practical use of Generative AI, API integration, FastAPI, frontend development, prompt engineering, and system testing. The application was tested locally to verify the major functionalities.

In the future, EduGenie can be enhanced with voice interaction, multilingual support, mobile applications, progress tracking, gamification, image/PDF input, and adaptive learning features. These improvements can make the system more useful and accessible for students.

Overall, EduGenie provides a simple platform that brings multiple learning-support features together in one application and can be further developed into a more comprehensive AI learning assistant.

Submitted by,

Team ID: SWTID-2026-6471

Team Lead: Poojadevi M

Team Members: Nathiya C
 Luvina R

Date: 03 /10/2026